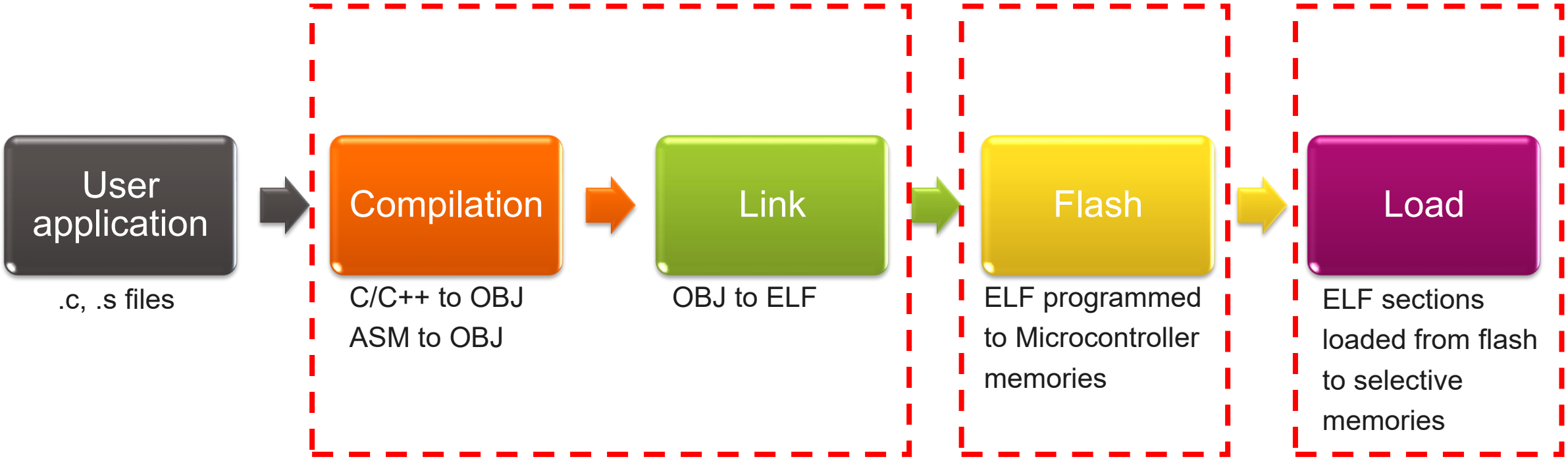
LINKER SCRIPT

Prakash Balasubramanian
Boya Vinay Kumar

11 Oct 2025

Source to binary - Flow

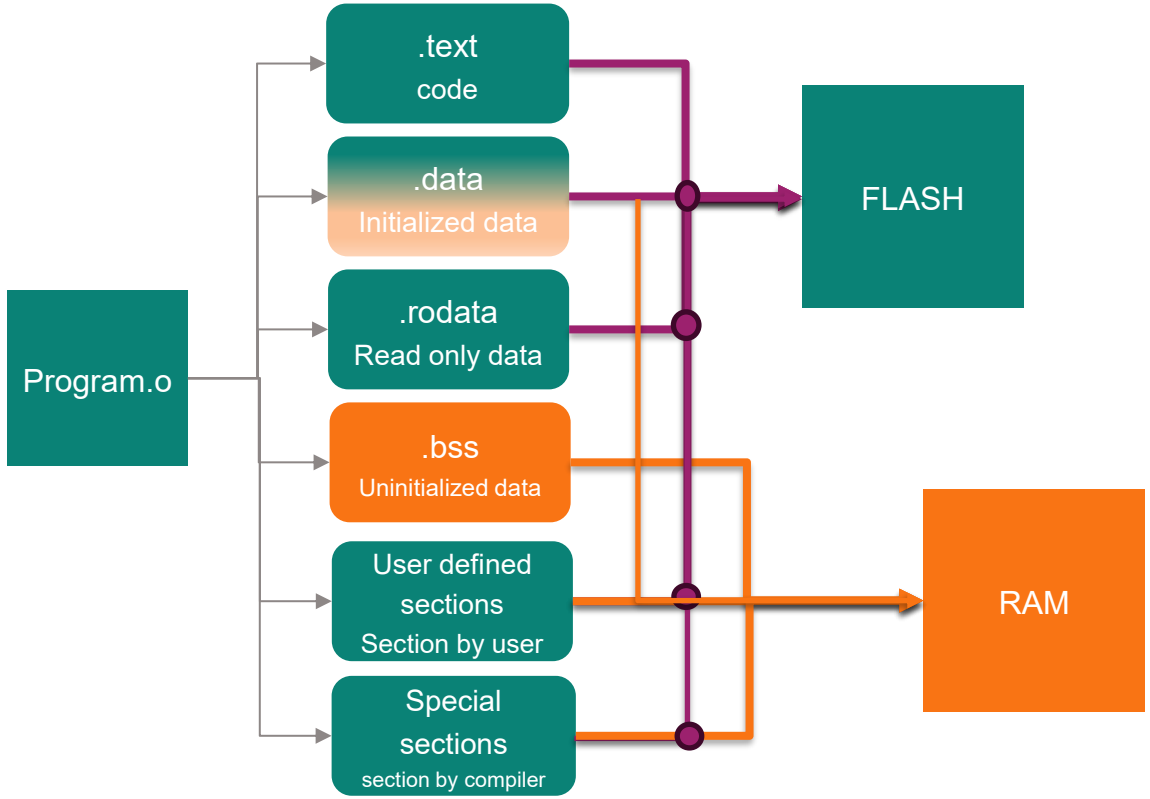


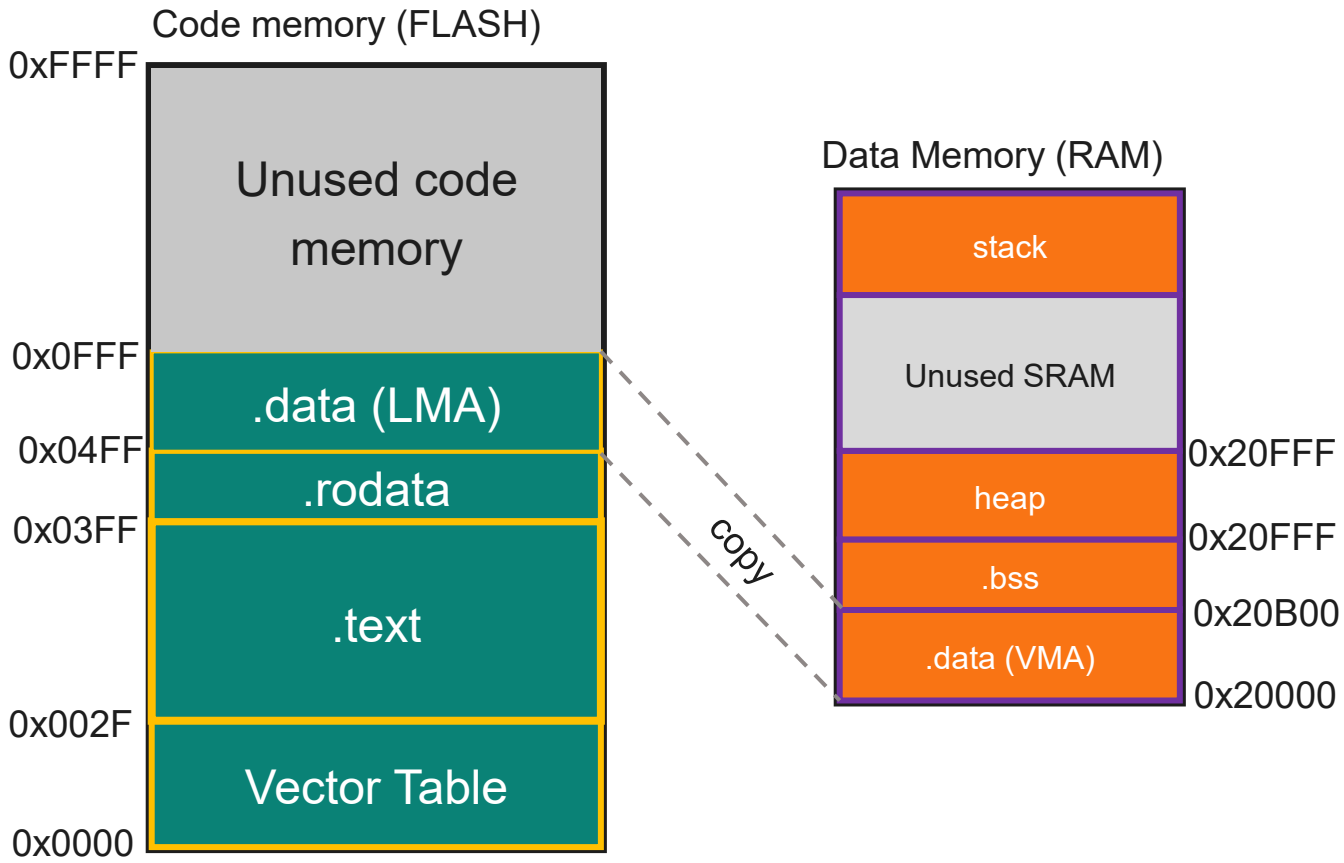
Compiler tools

Debugger / programmer

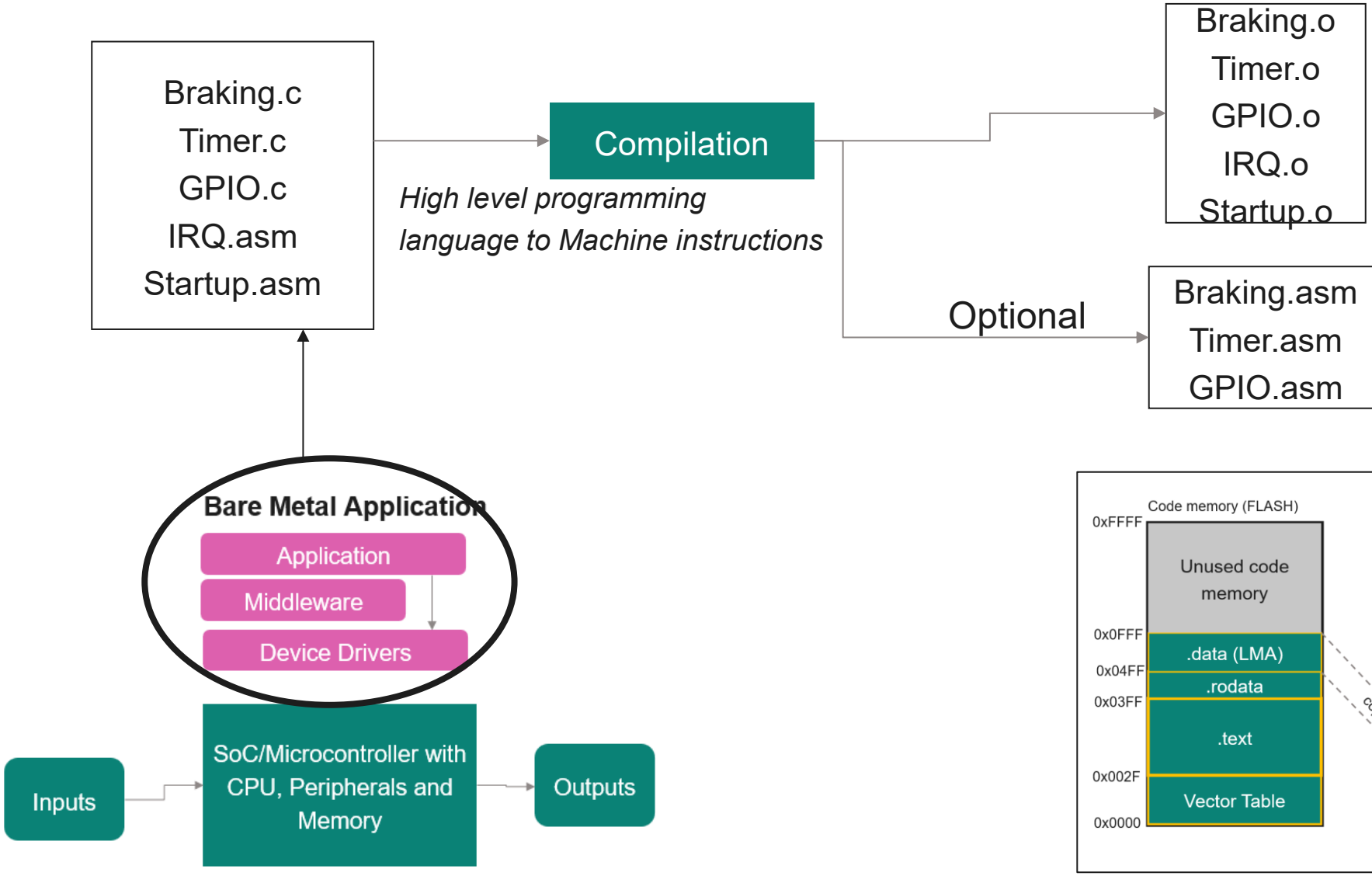
Microcontroller

Sections and its memory!





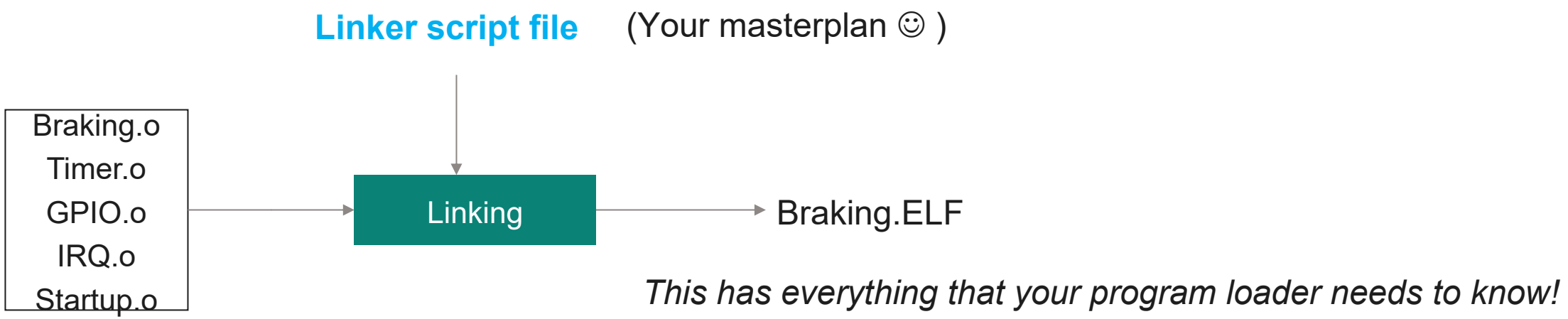
Source code compilation



The instructions and data in these object files are more or less ready for flashing.

BUT where must they be stored? What should be their address in memory?

Object linking



This has everything that your program loader needs to know!

Before Linking

func1()
func2()
func3()
Data1
Data2

Linking

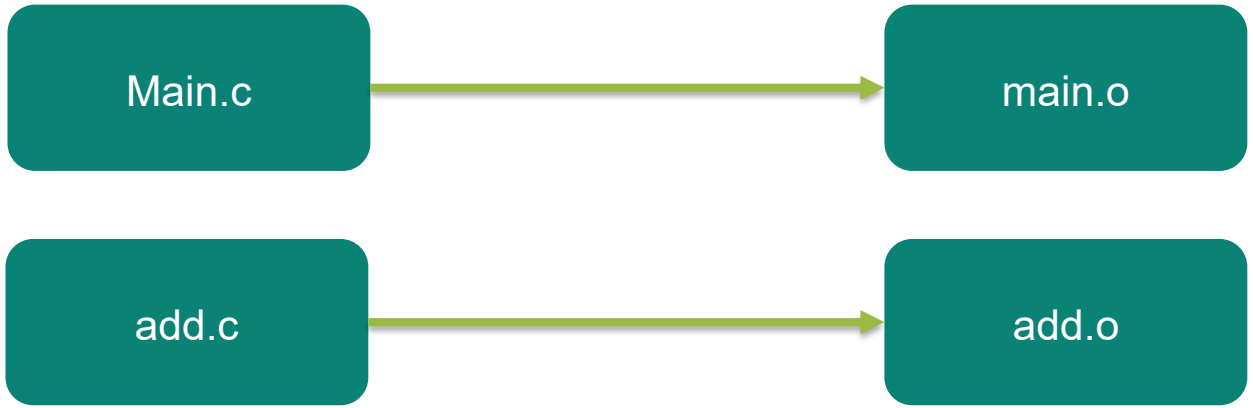
After Linking

func1() 0x20000010 – 0x20001FE
func2() 0x20000200 – 0x200005CB
func3() 0x200005D0 – 0x200005F0
Data1 0x80000000
Data2[8] 0x80000004 – 0x80000023

You specified the range in the linker script!

The linker has only followed your instructions faithfully.

Sections of program in ELF format



final.elf

.text(code sections from main.o and add.o)

. data(data sections from main.o and add.o)

. bss (bss sections from main.o and add.o)

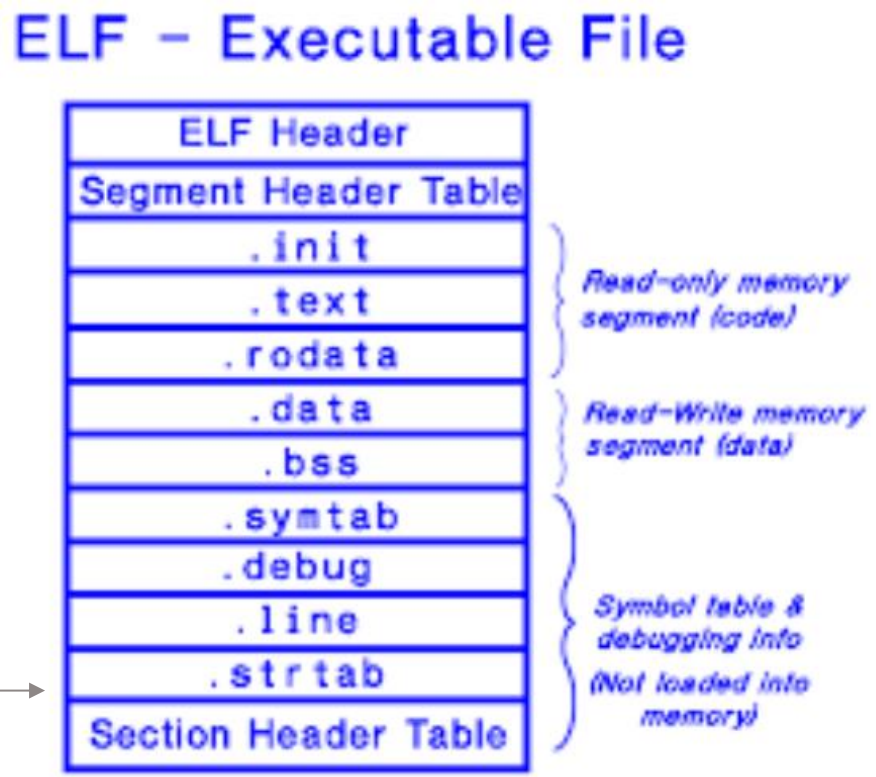
. rodata(const data sections from main.o and add.o)

What is an ELF anyways?

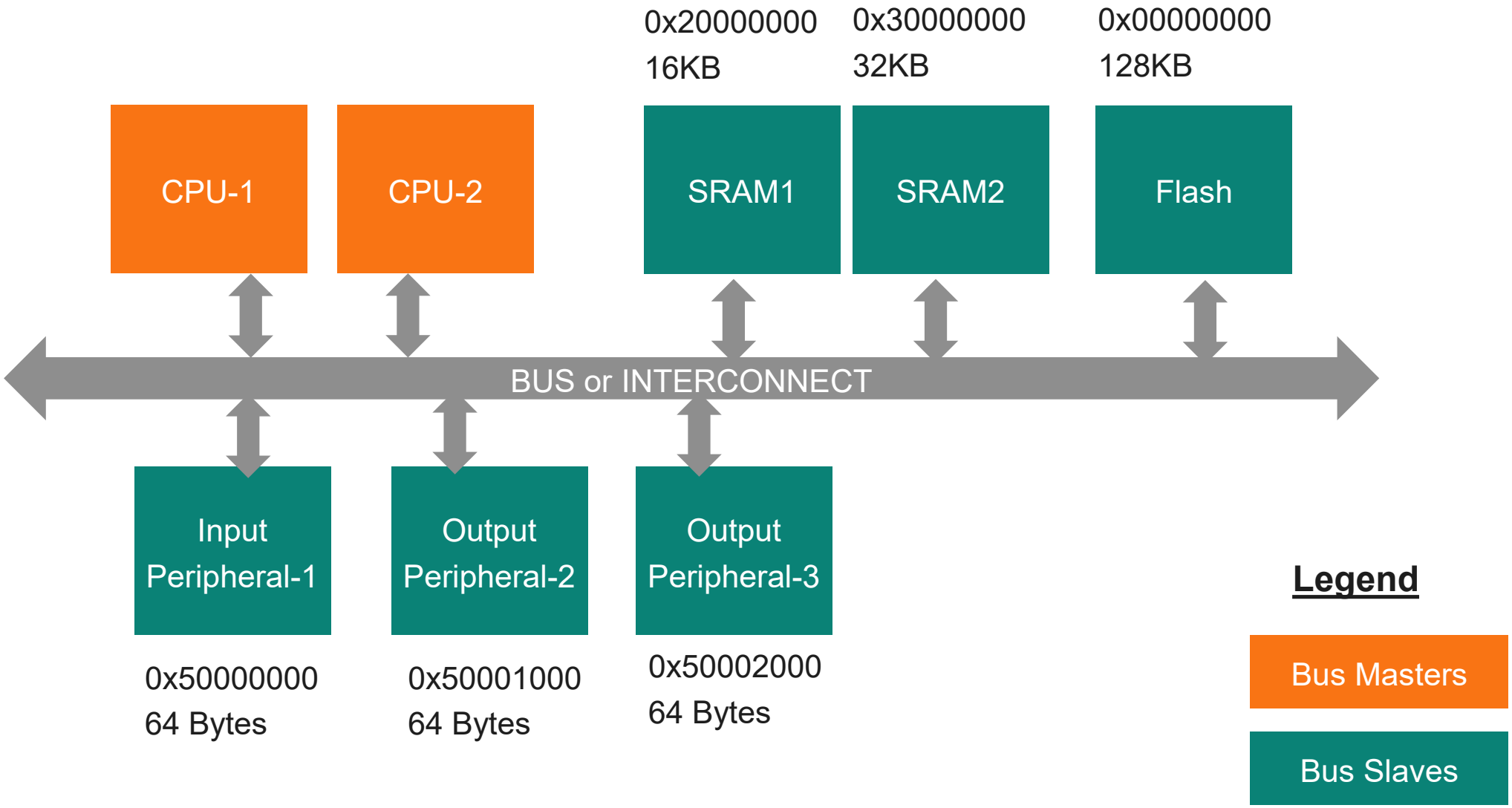
This bloke is the ELF!

This is also an ELF!

Executable and Linkable Format



An exemplary Microcontroller



Linker script highlights

A simple text file with a `.ld` extension.

Written using GNU linker commands.

Contains memory address and size information for code and data.

Specifies how object file sections are merged.

Defines memory placement of new sections in memory.

Included using `-T` option with the `arm-none-eabi-gcc` command.

Comman Linker commands:

- ENTRY
- MEMORY
- SECTIONS
- KEEP
- ALIGN
- AT>

arm-none-eabi-gcc -T <linker.ld > <flags> <object files> -o <destination>.elf

Ex: arm-none-eabi-gcc -T linker_script.ld main.o add.o -nostartfiles -o application.elf

Linker script – Big picture

Define MEMORY regions – i.e. Define start address and length of each system memory block

Assign your program to “Code”, “RODATA”, “DATA” and “BSS” INPUT SECTIONS respectively

Group several INPUT Sections with a common requirement to an OUTPUT section

Assign OUTPUT Sections to MEMORY sections

Example Linker Script (GNU syntax)

Define MEMORY regions

```
MEMORY
  {
     name [( attr )] : ORIGIN = origin , LENGTH = len
...
  }
```

```
MEMORY
{
  SRAM1(rwx): ORIGIN: 0x20000000, LENGTH = 32K

}
```

Example Linker Script (GNU syntax)

Define Sections

```
SECTIONS
{
  Output_Section1:
  {
     Object_File(Input section of the object file)
  } > REGION

 Output_Section2:
  {
    Object_File(Input section of the object file)
  } > REGION

}
```

```
SECTIONS
{
  .fasttext:
  {
     Adc.o(.text)
  } > SRAM1
  .data:
   {
      *(.data)
      *(.bss)
   } > SRAM2
 .text:
  {
    *(.text)
    *(.RODATA)
  } > FLASH
}
```

Linker Script!

linker_script.ld

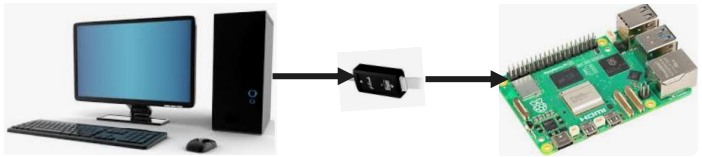
```
__FLASH_START = 0x00000000;
__FLASH_SIZE =  0x00020000;

__RAM_START = 0x20000000;
__RAM_SIZE =  0x00004000; /* 16KB */

__STACK_SIZE = 0x00001000; /* 4KB */
__HEAP_SIZE = 0x00000200; /* 512B */

ENTRY(Reset_Handler)

MEMORY
{
  FLASH (rx)  : ORIGIN = __FLASH_START, LENGTH = __FLASH_SIZE
  RAM   (rwx) : ORIGIN = __RAM_START,   LENGTH = __RAM_SIZE
}

__STACK_START = __RAM_START + __RAM_SIZE;
__STACK_END = __STACK_START - __STACK_SIZE;

__HEAP_START = __STACK_END - __HEAP_SIZE;
__HEAP_END = __STACK_END;

SECTIONS
{
    .text :
    {
        KEEP(*(.isr_vector))
        *(.text*)
        . = ALIGN(4);
        *(.rodata .rodata.*)
        . = ALIGN(4);
        __etext = .;
    } > FLASH

    _la_data = LOADADDR(.data);
    .data :
    {
        __data_start = .;
        *(.data*)
        . = ALIGN(4);
        __data_end = .;

    } > RAM AT> FLASH

    .bss :
    {
        __bss_start__ = .;
        *(.bss)
        *(.bss.*)
        . = ALIGN(4);
        __bss_end__ = .;
        . = ALIGN(4);
        end = .;
        __end__ = .;
    } > RAM

    .bss :
    {
        __bss_start__ = .;
        *(.bss)
        *(.bss.*)
        . = ALIGN(4);
        __bss_end__ = .;
        . = ALIGN(4);
        end = .;
        __end__ = .;
    } > RAM

    /* Dummy section for stack */
    Heap (NOLOAD) :
    {
        . = ABSOLUTE (__HEAP_START);
        *(.heap)
        . = ALIGN(4);

    } > RAM

    /* Dummy section for stack */
    Stack (NOLOAD) :
    {
        . = ABSOLUTE (__STACK_END);
        *(.stack)
    } > RAM
}
```

commands

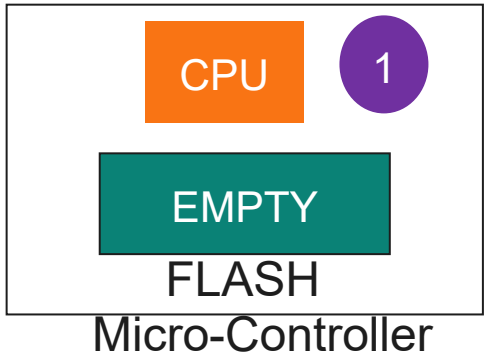
- arm-none-eabi-gcc -c main.c -mcpu=cortex-m0plus -o main.o
- arm-none-eabi-gcc -c add.c -mcpu=cortex-m0plus -o add.o
- arm-none-eabi-gcc -o application.elf *.o -T linker_script.ld -nostartfiles "-Wl,-Map=application.map“

- arm-none-eabi-objdump -h application.elf -> sections
- arm-none-eabi-objdump -s application.elf -> contents
- arm-none-eabi-readelf -a application.elf -> content of elf

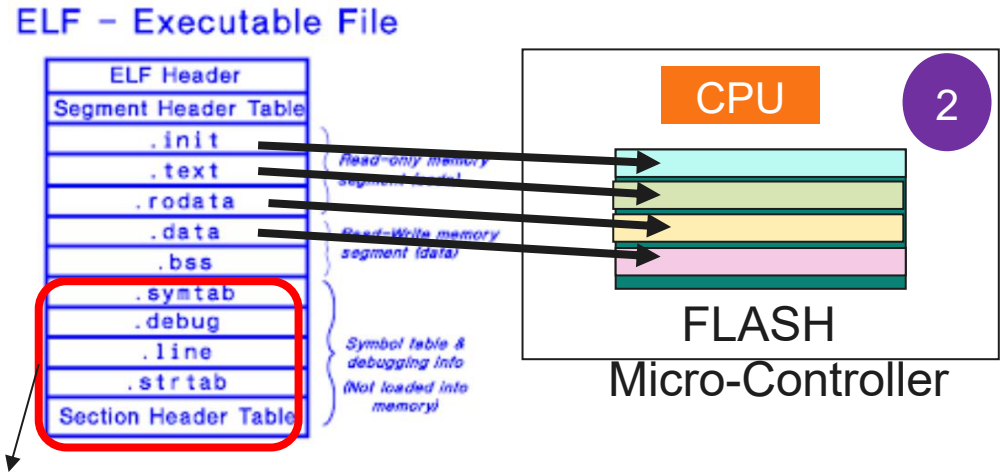
Putting them all together



Your brand new development board



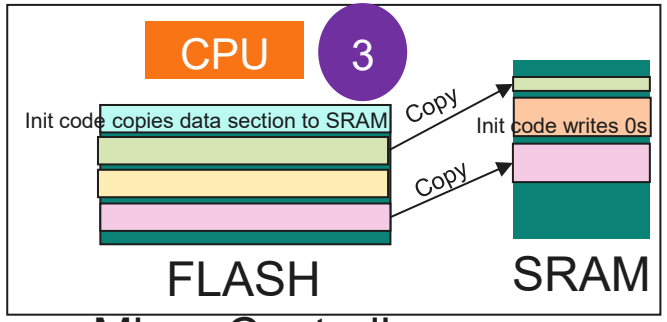
After Flashing your application



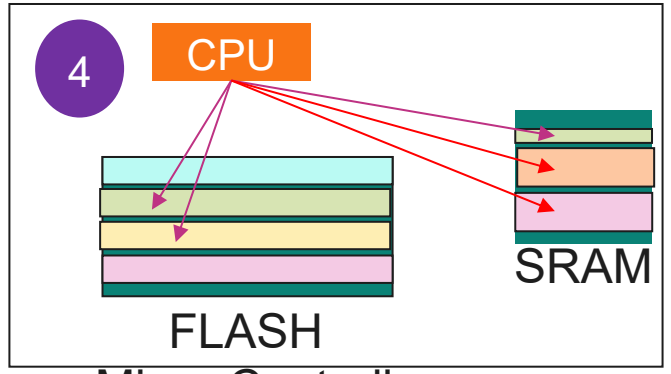
Not flashed.

These are needed only for debugging

At run time (After powering up)



At run time (After program loading)



CPU fetches TEXT and RODATA from flash

CPU accesses DATA and BSS in SRAM

CPU may fetch instructions of time critical TEXT segment which has been coped to SRAM.


Infineon